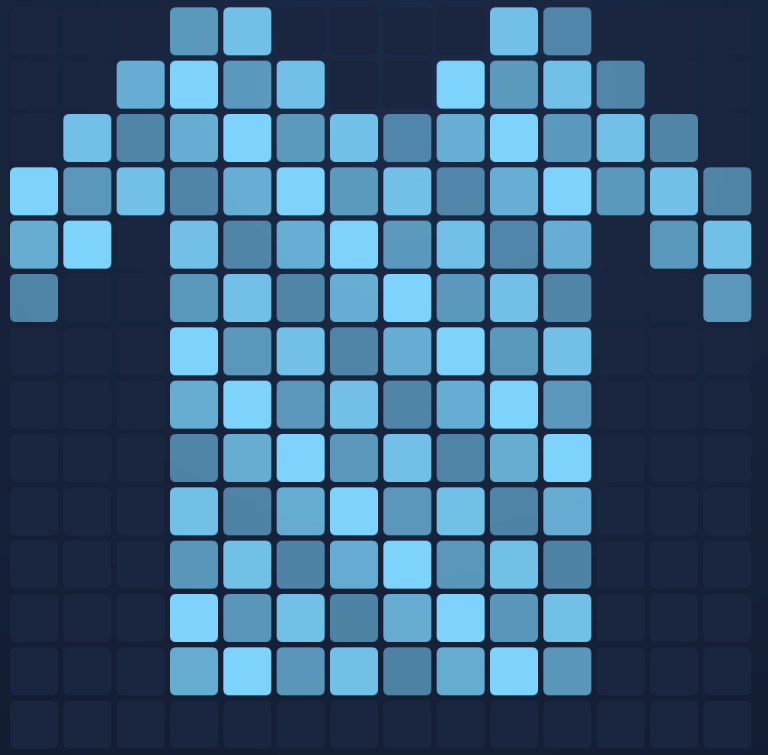
ECM963 · PESQUISA APLICADA COM IA

Fashion-MNIST na NPU

Um histórico de inferências no celular, com FlatList

André Maiolini - 19.02012-0



Uma NPU, mais de uma rede

O TCC

Processador RISC-V em FPGA com uma NPU, um circuito que acelera redes neurais. Até agora, ela só foi demonstrada reconhecendo dígitos (MNIST).

O problema

Na tela do computador, cada resultado substitui o anterior. Quem assiste não vê o histórico nem onde a rede mais erra.

Esta PoC: uma segunda rede, que reconhece peças de roupa (Fashion-MNIST), e um histórico dos resultados no celular.

A pergunta ficou mais precisa

INICIAL

Como mostrar no celular um histórico que cresce a cada inferência, sem travar a rolagem com mais de 50 itens?

REFINADA

A **FlatList** basta para esse histórico, com resultados chegando um a um e mais de 50 itens, ou é preciso partir para a **FlashList**?

O mapa com duas IAs e a documentação oficial levaram a FlatList para dentro da pergunta. E a resposta pode ser “não”: o teste decide.

Duas IAs, duas recomendações

Gemini
Recomendou a FlashList

ChatGPT
Recomendou a FlatList

Documentação
Não decide: depende de testar

Opção	Expo Go	Decisão
ScrollView	Sim	Não: desenha todos os itens de uma vez
FlatList	Sim	Sim: já vem no React Native e foi vista em aula
FlashList	Sim	Plano B, se a FlatList travar

Do toque à lista

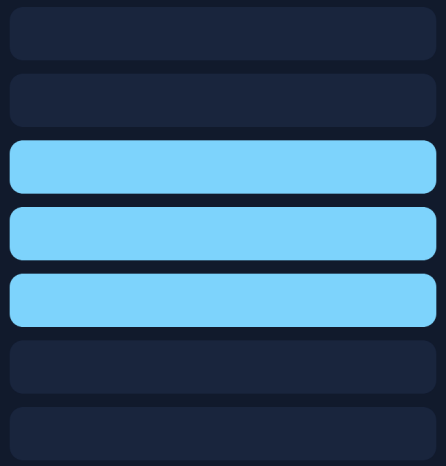
Celular
Sortear · Inferir · Rodar 20

→ Wi-Fi

Computador
Servidor que fala com a placa

→ UART

NPU na FPGA
Placa real (--placa) ou emulação



A FlatList só desenha o que está perto da tela

Em azul: os itens montados agora. Os de cinza saem e voltam quando você rola, por isso a lista continua leve mesmo crescendo.

04 · DEMONSTRAÇÃO

Ao vivo

Critério por critério, no celular

1 · Lista vazia

2 · Sortear → Inferir

3 · Rodar 20

4 · 60 itens rolando

5 · Servidor fora

RESULTADO

A FlatList bastou

60 itens, rolando até o fim e de volta, sem travar e sem espaços em branco, inclusive com resultados chegando durante a rolagem.

Não foi preciso trocar pela FlashList. Testado no celular, pelo Expo Go em modo de desenvolvimento, que é mais lento que o app final.

05 · ONDE A IA ERROU

Quatro erros, quatro formas de perceber

Gemini · pesquisa

Disse que a FlashList precisa do tamanho fixo dos itens

Percebi: lendo a página de mudanças da versão 2
Certo: isso só valia na versão antiga

Gemini · pesquisa

Atribuiu a configuração recyclingKey à FlashList

Percebi: procurando na documentação
Certo: ela é do expo-image

Agente · código

Mexeu no package.json sem ser pedido: o ESLint entrou e depois ele tentou tirar

Percebi: lendo o diff antes de aceitar
Certo: manter o ESLint; recusei a remoção

Agente · código

Montou a tela sem a área segura: a lista ficava atrás dos botões do Android

Percebi: testando no celular, não no chat
Certo: SafeAreaView do react-native-safe-area-context

Próximos passos

01 Rodar na FPGA real

Comparar placa e emulação em 200 imagens. Até agora, o modo placa só foi testado com uma placa simulada.

antes da banca

02 Medir do celular até a placa

Somar Wi-Fi, servidor e UART e comparar com os ~9 ms da UART, o gargalo já apontado no TCC.

antes da banca

03 Usar o app na banca do TCC

Mostrar a mesma NPU com uma segunda rede, com o histórico na tela.

na banca

Fora do plano: salvar o histórico entre sessões e gerar uma development build (que corrigiria a barra do Android).

FONTES

Documentação consultada

Assunto	Página
FlatList	reactnative.dev/docs/flatlist
Otimizar a FlatList	reactnative.dev/docs/optimizing-flatlist-configuration
ScrollView	reactnative.dev/docs/scrollview
FlashList no Expo	docs.expo.dev/versions/latest/sdk/flash-list
FlashList v2	shopify.github.io/flash-list/docs/v2-migration
expo-image	docs.expo.dev/versions/latest/sdk/image
expo-haptics	docs.expo.dev/versions/v58.0.0/sdk/haptics

Diário de pesquisa e critérios: PESQUISA.md e CRITERIOS.md no repositório do projeto.